

CHƯƠNG II – DANH SÁCH LIÊN KẾT

LẬP TRÌNH NÂNG CAO

Giảng viên: Lê Văn Hùng

 0906238311

 hunglv@hvnh.edu.vn

1. Khái niệm

Danh sách liên kết đơn là một tập hợp các Node được phân bố động, được sắp xếp theo cách sao cho mỗi Node chứa một giá trị (**Data**) và một con trỏ (**Next**). Con trỏ sẽ trỏ đến phần tử kế tiếp của danh sách liên kết đó. Nếu con trỏ mà trỏ tới **NULL**, nghĩa là đó là phần tử cuối cùng của **linked list**.

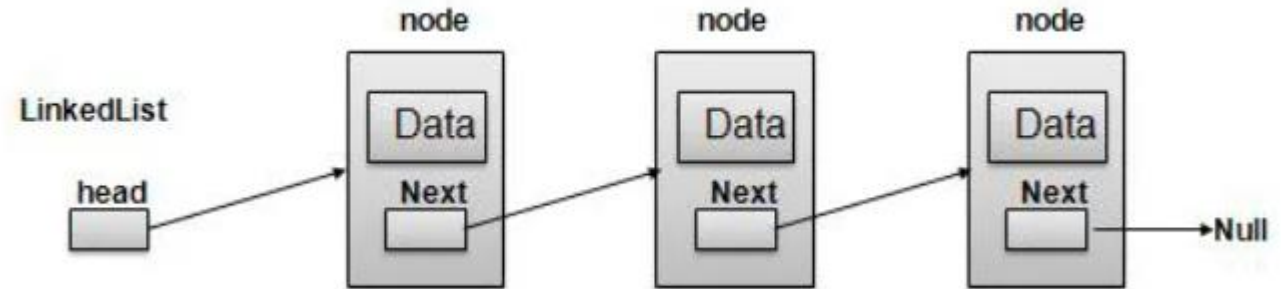
Mỗi Node trong danh sách liên kết đơn được chia thành hai thành phần chính đó là:

- **Phần data:** lưu trữ dữ liệu của node
- **Phần liên kết:** Lưu trữ địa chỉ của phần tử kế tiếp trong danh sách, hoặc lưu trữ giá trị NULL nếu là phần tử cuối cùng trong danh sách.



1. Khái niệm

Danh sách liên kết đơn là một tập hợp các Node được phân bố động, được sắp xếp theo cách sao cho mỗi Node chứa một giá trị (**Data**) và một con trỏ (**Next**). Con trỏ sẽ trỏ đến phần tử kế tiếp của danh sách liên kết đó. Nếu con trỏ mà trỏ tới **NULL**, nghĩa là đó là phần tử cuối cùng của **linked list**.



1. Khái niệm

- ❖ Danh sách các kiểu danh sách liên kết:
- ❖ **Danh sách liên kết đơn**(Single linked list): Chỉ có sự kết nối từ phần tử phía trước tới phần tử phía sau.
- ❖ **Danh sách liên kết đôi**(Double linked list): Có sự kết nối 2 chiều giữa phần tử phía trước với phần tử phía sau
- ❖ **Danh sách liên kết vòng**(Circular Linked List): Có thêm sự kết nối giữa 2 phần tử đầu tiên và phần tử cuối cùng để tạo thành vòng khép kín.

1. Khái niệm

Nội dung	Mảng	Danh sách liên kết
Kích thước	•Kích thước cố định •Cần chỉ rõ kích thước trong khi khai báo	•Kích thước thay đổi trong quá trình thêm/ xóa phần tử •Kích thước tối đa phụ thuộc vào bộ nhớ
Cấp phát bộ nhớ	•Tĩnh: Bộ nhớ được cấp phát trong quá trình biên dịch	•Động: Bộ nhớ được cấp phát trong quá trình chạy
Thứ tự & sắp xếp	•Được lưu trữ trên một dãy ô nhớ liên tục	•Được lưu trữ trên các ô nhớ ngẫu nhiên
Truy cập	•Truy cập tới phần tử ngẫu nhiên trực tiếp bằng cách sử dụng chỉ số mảng: $O(1)$	•Truy cập tới phần tử ngẫu nhiên cần phải duyệt từ đầu/cuối đến phần tử đó: $O(n)$

2. Cài đặt danh sách liên kết

```
0
1 struct LinkedList{
2     int data;
3     struct LinkedList *next;
4 };
5
```

Khai báo trên sẽ được sử dụng cho mọi Node trong linked list. Trường `data` sẽ lưu giữ giá trị và `next` sẽ là con trỏ để trỏ đến thằng kế tiếp của nó.

Tại sao next lại là kiểu `LinkedList` của chính nó? Bởi vì nó là con trỏ trỏ của chính bản thân nó, và nó trỏ tới một thằng Node kế tiếp cũng có kiểu `LinkedList`.

Tạo mới một danh sách liên kết rỗng

```
0  
1 node InitHead(){  
2     node head;  
3     head = NULL;  
4     return head;  
5 }  
6
```

Tạo mới một danh sách liên kết

```
0
1 typedef struct LinkedList *node; //Từ giờ dùng kiểu dữ liệu LinkedList có thể thay bằn
2
3 node CreateNode(int value){
4     node temp; // declare a node
5     temp = (node)malloc(sizeof(struct LinkedList)); // Cấp phát vùng nhớ dùng malloc()
6     temp->next = NULL; // Cho next trỏ tới NULL
7     temp->data = value; // Gán giá trị cho Node
8     return temp; // Trả về node mới đã có giá trị
9 }
10
```

Lưu ý: Không giống với mảng, cần khai báo *arr[size]*. Trong linked list, vì mỗi Node sẽ có con trỏ liên kết đến Node tiếp theo. Do đó, với danh sách liên kết đơn, bạn chỉ cần lưu giữ Node đầu tiên(HEAD). Có head rồi bạn có thể đi tới bất cứ Node nào.

- **malloc** là hàm cấp phát bộ nhớ của C. Với C++ chúng ta dùng **new**
- **sizeof** là hàm trả về kích thước của kiểu dữ liệu, dùng làm tham số cho hàm *malloc*

Thêm Node vào danh sách liên kết

Thêm vào cuối danh sách:

- ❖ **Trường hợp 1:** Nếu danh sách liên kết đơn rỗng thì node mới được xem là node đầu tiên và cũng là node cuối cùng.
- ❖ **Trường hợp 2:** Nếu danh sách liên kết đơn không rỗng thì:
 - Cho con trỏ liên kết (next) của node cuối danh sách hiện tại trỏ đến node mới.
 - Cho con trỏ cuối của danh sách liên kết đơn (*tail) trỏ vào node mới.

Thêm Node vào danh sách liên kết

```
0
1 node AddTail(node head, int value){
2     node temp,p;// Khai báo 2 node tạm temp và p
3     temp = CreateNode(value);//Gọi hàm createNode để khởi tạo node temp có next trỏ tới NULL
4     if(head == NULL){
5         head = temp;    //Nếu linked list đang trống thì Node temp là head luôn
6     }
7     else{
8         p = head;// Khởi tạo p trỏ tới head
9         while(p->next != NULL){
10             p = p->next;//Duyệt danh sách liên kết đến cuối. Node cuối là node có next trỏ tới NULL
11         }
12         p->next = temp;//Gán next của thằng cuối = temp. Khi đó temp sẽ là thằng cuối(đang thiếu dấu chấm phẩy)
13     }
14     return head;
15 }
16
```

Thêm vào vị trí đầu danh sách

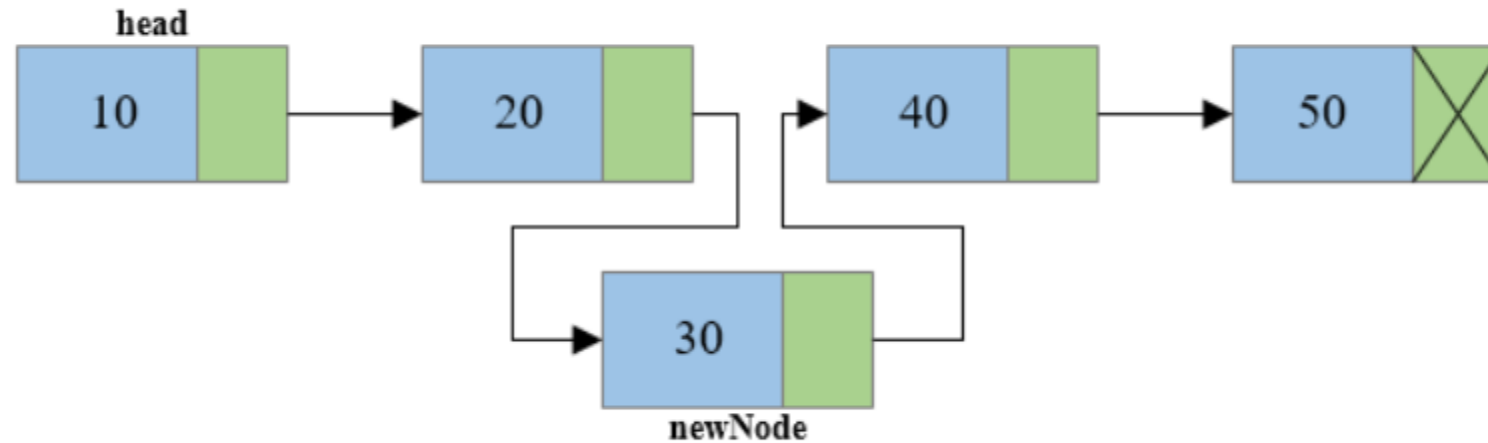
Việc thêm vào đầu chính là việc cập nhật lại thẳng head. Ta gọi Node mới (temp), ta có:

- Nếu head đang trỏ tới NULL, nghĩa là linked list đang trống, Node mới thêm vào sẽ làm head luôn
- Ngược lại, ta phải thay thế thẳng head cũ bằng head mới. Việc này phải làm theo thứ tự như sau:
 - Cho next của temp trỏ tới head hiện hành
 - Đặt temp làm head mới

```
0
1 node AddHead(node head, int value){
2     node temp = CreateNode(value); // Khởi tạo node temp với data = value
3     if(head == NULL){
4         head = temp; // //Nếu linked list đang trống thì Node temp là head luôn
5     }else{
6         temp->next = head; // Trỏ next của temp = head hiện tại
7         head = temp; // Đổi head hiện tại = temp(Vì temp bây giờ là head mới mà)
8     }
9     return head;
10 }
11
```

Thêm vào vị trí bất kỳ

Thêm vào vị trí bất kỳ



Thêm Node vào giữa danh sách liên kết

Để làm được
khi đó ta cần

- Cho node
- Cho Node

Lưu ý: Chỉ

```
0
1 node AddAt(node head, int value, int position){
2     if(position == 0 || head == NULL){
3         head = AddHead(head, value); // Nếu vị trí chèn là 0, tức là thêm vào đầu
4     }else{
5         // Bắt đầu tìm vị trí cần chèn. Ta sẽ dùng k để đếm cho vị trí
6         int k = 1;
7         node p = head;
8         while(p != NULL && k != position){
9             p = p->next;
10            ++k;
11        }
12
13        if(k != position){
14            // Nếu duyệt hết danh sách lk rồi mà vẫn chưa đến vị trí cần chèn, ta sẽ m
15            // Nếu bạn không muốn chèn, hãy thông báo vị trí chèn không hợp lệ
16            head = AddTail(head, value);
17            // printf("Vi tri chen vuot qua vi tri cuoi cung!\n");
18        }else{
19            node temp = CreateNode(value);
20            temp->next = p->next;
21            p->next = temp;
22        }
23    }
24    return head;
25 }
26
```

sử là Node Q,

Xóa một nút khỏi danh sách_Xóa đầu

- ❖ Xóa đầu đơn giản lắm, bây giờ chỉ cần cho thằng kế tiếp của head làm head là được thôi. Mà thằng kế tiếp của head chính là *head->next*.

```
0
1 node DelHead(node head){
2     if(head == NULL){
3         printf("\nChà có gì để xóa hết!");
4     }else{
5         head = head->next;
6     }
7     return head;
8 }
9
```

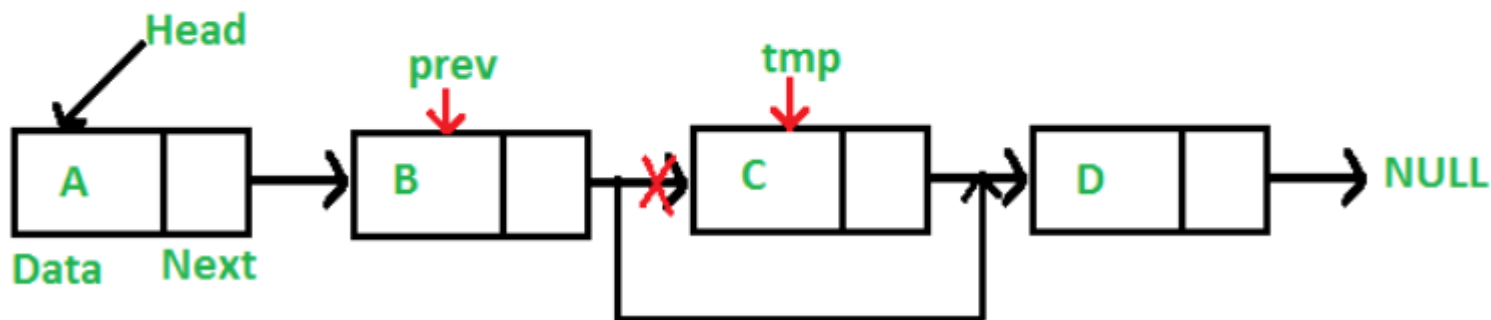
Xóa một nút khỏi danh sách_Xóa cuối

- ❖ Xóa cuối mới nhọc nè, nhọc ở chỗ phải duyệt đến thẳng cuối – 1, cho next của cuối – 1 đó bằng NULL.

```
0
1 node DelTail(node head){
2     if (head == NULL || head->next == NULL){
3         return DelHead(head);
4     }
5     node p = head;
6     while(p->next->next != NULL){
7         p = p->next;
8     }
9     p->next = p->next->next; // Cho next bằng NULL
10    // Hoặc viết p->next = NULL cũng được
11    return head;
12 }
```

Xóa ở vị trí bất kỳ

Việc xóa ở vị trí bất kỳ cũng khá giống xóa ở cuối kia. Đơn giản là chúng ta bỏ qua một phần tử, như ảnh sau:



Xóa ở vị trí bất kỳ

```
0
1 node DelAt(node head, int position){
2     if(position == 0 || head == NULL || head->next == NULL){
3         head = DelHead(head); // Nếu vị trí chèn là 0, tức là thêm vào đầu
4     }else{
5         // Bắt đầu tìm vị trí cần chèn. Ta sẽ dùng k để đếm cho vị trí
6         int k = 1;
7         node p = head;
8         while(p->next->next != NULL && k != position){
9             p = p->next;
10            ++k;
11        }
12
13        if(k != position){
14            // Nếu duyệt hết danh sách lk rồi mà vẫn chưa đến vị trí cần chèn, ta sẽ r
15            // Nếu bạn không muốn xóa, hãy thông báo vị trí xóa không hợp lệ
16            head = DelTail(head);
17            // printf("Vi tri xoa vuot qua vi tri cuoi cung!\n");
18        }else{
19            p->next = p->next->next;
20        }
21    }
22    return head;
23 }
```

Lấy giá trị ở vị trí bất kỳ

- ❖ Chúng ta sẽ viết một hàm để truy xuất giá trị ở chỉ số bất kỳ nhé. Trong trường hợp chỉ số vượt quá chiều dài của linked list – 1, hàm này trả về vị trí cuối cùng. Do hạn chế là chúng ta không thể raise error khi chỉ số không hợp lệ. Tôi mặc định chỉ số bạn truyền vào phải là số nguyên không âm. Nếu bạn muốn kiểm tra chỉ số hợp lệ thì nên kiểm tra trước khi gọi hàm này.

```
int Get(node head, int index){  
    int k = 0;  
    node p = head;  
    while(p->next != NULL && k != index){  
        ++k;  
        p = p->next;  
    }  
    return p->data;  
}
```

Tìm kiếm trong danh sách

Hàm tìm kiếm này sẽ trả về chỉ số của Node đầu tiên có giá trị bằng với giá trị cần tìm. Nếu không tìm thấy, chúng ta trả về -1.

```
0
1 int Search(node head, int value){
2     int position = 0;
3     for(node p = head; p != NULL; p = p->next){
4         if(p->data == value){
5             return position;
6         }
7         ++position;
8     }
9     return -1;
10 }
11
```

Duyệt danh sách

Việc duyệt danh sách liên kết cực đơn giản. Khởi tạo từ Node head, bạn cứ thế đi theo con trỏ next cho tới trước khi Node đó NULL.

```
0
1 void Traverser(node head){
2     printf("\n");
3     for(node p = head; p != NULL; p = p->next){
4         printf("%5d", p->data);
5     }
6 }
7
```

Hàm nhập danh sách liên kết

```
node Input(){
    node head = InitHead();
    int n, value;
    do{
        printf("\nNhap so luong phan tu n = ");
        scanf("%d", &n);
    }while(n <= 0);

    for(int i = 0; i < n; ++i){
        printf("\nNhap gia tri can them: ");
        scanf("%d", &value);
        head = AddTail(head, value);
    }
    return head;
}
```

Hàm đếm số phần tử của danh sách

```
int Length(node head){  
    int length = 0;  
    for(node p = head; p != NULL; p = p->next){  
        ++length;  
    }  
    return length;  
}
```